

- 1) Nella gerarchia **Shape**, **Circle**, **Rectangle**, **Square** dell'esercitazione precedente, introdurre una interfaccia **Scalable** che dichiara un metodo *scale(double factor)*. Il metodo consente di modificare le dimensioni dei lati/raggio di una figura in base al fattore di proporzionalità *factor*.
Introdurre anche l'interfaccia **Drawable** che dichiara un metodo *draw()* per visualizzare i dati dell'oggetto. Rendere **Shape** classe astratta e verificare che per questa classe non si possono istanziare oggetti.
- 2) Utilizzate quindi le vostre classi per un gestore di schermo che memorizza figure geometriche in una lista di Shape, e che ha funzionalità per aggiungerne, rimuoverne, ridisegnare tutto e scalare tutti gli oggetti. Scrivete infine il main che crea un oggetto gestore di schermo e gestisce l'interazione con l'utente.
- 3) Considerato il seguente codice:

```
class Animale {}  
interface Felino {}  
class Leone {}
```

Se volessimo mettere in relazione di ereditarietà i tipi precedenti, quali dei seguenti snippet sono validi dal punto di vista del compilatore?

1. class Animale extends Felino {}
 2. interface Felino extends Animale {}
 3. class Leone extends Felino {}
 4. class Leone extends Animale implements Felino {}
 5. class Animale extends Leone implements Felino {}
- 4) Dato che un'interfaccia può dichiarare oltre a metodi astratti, anche metodi statici pubblici e privati, e implementati (di default) pubblici e privati, e visto che con le interfacce possiamo implementare l'ereditarietà multipla, perché mai dovremmo preferire una classe astratta ad un'interfaccia?

ESERCIZI DEL LIBRO DI TESTO

Esercizio 8.5

Spiegare le differenze tra override, hiding e overload?

Esercizio 8.15

Descrivere i contratti dei metodi equals e hashCode.

Implementare i metodi equals ed hashCode per le classi della gerarchia delle figure geometriche e verificarne il corretto funzionamento.

Esercizio 8.26 Scrivania

Implementare una classe Scrivania che possa contenere oggetti che si possono posare su di essa. Questo significa che bisogna creare un'interfaccia sealed Appoggiabile senza alcun metodo. Poi estendere questa classe con i record Libro, Computer, Lampada, Penna e PortaPenne ognuna delle quali con le proprie caratteristiche. Creare la classe Studente, che definisce un metodo usa che prende in input un oggetto Scrivania, cicla sugli oggetti appoggiati su di essa e tramite il pattern matching stampi per ogni oggetto le caratteristiche che gli interessano. Testare tutto con una classe chiamata TestScrivania.

Esercizio 7.18 Dati Immobiliare

Creare una classe Jmmobiliare che definisce i metodi inserisciAppartamento, inserisciGarage e inserisciPalazzo, a cui passare i relativi oggetti. Questi metodi per ora si limiteranno a stampare il metodo toString degli oggetti che vengono creati. Come sempre, creare una classe di test chiamata TestJmmobiliare e testare i tre metodi.

Progetto8.1 Jmmobiliare (Versione1.4)

Partendo dalla soluzione del progetto 7.1, nella classe Jmmobiliare sostituire i metodi inserisciAppartamento, inserisciGarage e inserisciPalazzo con il metodo inserisciImmobile, che prende in input un immobile generico. Come sempre, creare una classe di test chiamata TestJmmobiliare e testare il nuovo metodo.